Verified Optimizations to Flock's RS Prover

measurements on Apple M1 Max, single-threaded; zerocheck at 2^{18}, PCS open at 2^{16} BLAKE3 compressions — and, from Section 9, the union protocol at m=32 (2^{18} compressions), multi- and single-threaded

branch `snark-fast-improvements`

August–September 2026

Abstract

Seven changes to the RS (additive-NTT) zerocheck path, each kept only after a paired, order-alternating A/B measurement with a decisive sign test. Together they reduce zerocheck round 1 from 1477 ms to 1205 ms (−18.4%), round 2 from 433 to 312 ms (−28%), the rounds-3+ tail from 455 to 307 ms (−33%), and whole-proof time by ≈ 11% single-threaded. Three are ideas ported from an independently optimized fork of this prover (the "challenge" tree), re-derived and re-implemented rather than copied; three originated here. The recurring themes: reductions in fields of characteristic two are $\mathbb{F}_2$-linear and can be deferred almost arbitrarily; work that a circuit fixes structurally can be skipped with a one-word guard; and on Apple silicon the boundary between the vector and general register files costs more than most instruction counts. A later section extends the record to the PCS open, whose dominant pass loses its per-slot field multiply to the same $\mathbb{F}_2$-linearity argument (§6.1). The final part (Section 9) records the campaign after the prover moved onto the repository's union protocol: which of the earlier wins survived the change of statement, which died with it, and the twelve that were landed on the new protocol — among them the statistics ladder in the open, the any-size Merkle leaf batcher, and the promotion of the AG zerocheck — which together took the m=32 prove from 854 to 438 ms multi-threaded and from 5.25 to 2.91 s single-threaded.

Preliminaries

Shared definitions and the benchmark shape. The numbered sections group the work by prover phase, in pipeline order; each subsection *is one verified optimization, presented with its mechanism and its paired measurement.*

Notation. A few recurring terms, used across multiple sections below:

- **witness builder** ("the builder"): the code that serializes the R1CS witness into the packed bit-buffers z, a, b.
- **drain** (the gather drain): the loop that walks the per-row lookup tables to accumulate round 1's output messages; detailed in §2.3.
- **lincheck stripe**: the repacking of the witness z, indexed by inner/outer position, that the lincheck phase consumes; defined fully in §2.2.

Fields. $\mathbb{F}_{2^8} = \mathbb{F}_2[x]/(x^8+x^4+x^3+x+1)$ and the GHASH field $\mathbb{F}_{2^{128}} = \mathbb{F}_2[X]/(X^{128}+X^7+X^2+X+1)$, with the ring embedding $\varphi : \mathbb{F}_{2^8} \hookrightarrow \mathbb{F}_{2^{128}}$. Write $\alpha := \varphi(x)$ and $\gamma := X$.

The witness is $z \in \mathbb{F}_2^{2^m}$; at the benchmark shape $m = 32$ (2^{18} BLAKE3 compressions, $k_{\log} = 14$ bits per block). After the univariate skip, round 1 of the zerocheck views the m index bits as

$$[\ \underbrace{\lambda}_{6\ \text{(univariate)}}\ \mid\ \underbrace{s}_{3\ \text{(small)}}\ \mid\ \underbrace{b}_{4\ \text{(medium)}}\ \mid\ \underbrace{t}_{m-13\ \text{(rest)}}\ ],$$

and must emit, for each of the 64 points λ of the skip domain, the quadratic message $P^{AB}(\lambda)$ and the linear message $P^{C}(\lambda)$, plus a pair of *banks* (the ring-switch helper $\hat{s}_{v,C}$) that keep the small parity dimension unfolded: the first small coordinate s_0 is *not* summed out of the C-side accumulation — round 1 emits one accumulator per value of s_0 (two banks) instead of a single combined value, because the c-claim's PCS opening needs the two halves separately ($\hat{s}_{v,C}$'s canonical form recombines them with weights 1 and α; the wire message is their plain sum).

The pinned values below are protocol design, common to both trees — not one of this document's optimizations. The optimization content is *which direction* a kernel exploits them; the two directions are stated in math at the end of this section. The protocol pins the small and medium challenges to *friendly* values: $r_{6+i} = \varphi(v_i)$ with $v = (\texttt{0xF7}, \texttt{0x53}, \texttt{0xB5})$, and $\beta_i = \gamma^{2^{i-1}}/(1+\gamma^{2^{i-1}})$. These make the eq weights geometric:

$$\prod_{i=0}^{2} \mathrm{eq}(r_{6+i}, K_i) \;=\; C_s\,\alpha^K, \qquad \prod_{i=1}^{4} \mathrm{eq}(\beta_i, b_i) \;=\; \gamma^b/D, \tag{1}$$

with $K = \sum K_i 2^i$, $C_s = \prod_i (1 + r_{6+i}) = \varphi(\texttt{0x1C})$, and $D = \prod_i (1 + \gamma^{2^{i-1}})$. The baseline kernels exploit (1) through lookup tables: the geometric weights are enumerated once into byte-indexed convert tables and applied by gathers. Three optimizations below exploit it in the opposite direction, replacing tables by folds at the actual challenge values: the C-side banks are recomputed as a true multilinear fold at the pinned challenges, with the over-folded eq factors undone by a closed-form constant that exists only because of (1) (§2.2); the prep kernel's weight x^K is factored so one piece stays table-driven while the x^2 piece is applied as a shift-and-conditional-XOR directly on the operand (§2.1); and the PCS open builds its composed fold table by walking the monomials $X^r e$ with 127 exact shift-and-folds instead of 128 general multiplies (§6.1 — resting on $\gamma = X$ being the generator rather than on (1) itself).

The two directions, in math. Both evaluate contractions against the same geometric weight vector: with $w_K = C\,\alpha^K$ for $K < n$ and bit-inputs $u \in \mathbb{F}_2^n$, the quantity is $\langle w, u\rangle = \sum_{K<n} u_K\, w_K$. The *table* direction precomputes the entire linear map once,

$$T[u] \;=\; \sum_{K<n} u_K\, w_K \quad \text{for all } u \in \mathbb{F}_2^n \qquad (2^n \text{ entries, built by subset-sum doubling}),$$

after which each of N streamed inputs costs a single gather, $\langle w, u^{(j)}\rangle = T[u^{(j)}]$: setup $O(2^n)$, then per-input cost independent of n and no multiplies at all. The *fold* direction never materializes T; it applies the weights level by level to the resident data itself,

$$u \;\leftarrow\; u_{\mathrm{even}} \;+\; \alpha^{2^i}\, u_{\mathrm{odd}}, \qquad i = 0, \ldots, \log_2 n - 1,$$

costing $n-1$ multiplies per resident vector and touching no table memory. Hence the rule separating them: the table amortizes when one fixed map meets a stream of many inputs

($N \gg 2^n/n$, the load-port-bound drains); the fold wins when there is essentially one resident object — an accumulator, a bank, or the table T itself while it is being constructed (subset-sum doubling *is* a fold; §6.1 is that observation at $n = 128$). Folds at freshly *sampled* challenges (rounds two and onward) get no such shortcut — there a fold is an ordinary multiply.

Measurement protocol. All numbers are single-threaded (ST) at the shape above unless marked 8T (eight threads). Each verdict is a paired A/B: both arms built once, one discarded warm-up each, then eight pairs with the *order alternating within each pair* (a fixed order is biased by thermal drift by roughly the size of the effects measured here). Phase times are the minimum over the ∼5 zerocheck invocations within one run; we call each phase's measured time slice its *bucket*. A change is kept only on a decisive sign test (a nonparametric test of whether the paired differences all share the same sign); two of the six wins were 8/8 with disjoint ranges. Measure only on AC power with the battery above ∼ 60%: low battery caps frequencies, and fast-charging a nearly empty battery is worse.

1 Witness generation

Everything downstream consumes the bit-packed witness; its witness builder (defined in Preliminaries) is the first timed phase.

1.1 Streaming full-word publication

Baseline: OR sub-word fields into pre-zeroed storage: a ≈130 MB×3 memset plus a read-modify-write on every store.

Improvement: A sequential writer that carries a partial word in a register publishes complete u64s in order and never reads the destination.

Mechanism: Eliminates the ≈130 MB×3 memset and the per-store read-modify-write entirely.

Result: ST 87–90 → 64.8–68.9 ms (−24%, 3/3 disjoint); 8T −20%.

The BLAKE3 witness builder fills three bit-packed buffers (z, a, b; one bit per R1CS wire) per compression block. The incumbent wrote them by OR-ing sub-word fields into pre-zeroed storage: a ≈130 MB ×3 memset, then a read-modify-write on every store. But the circuit's rows are *contiguous* through the block's useful bits, so a sequential writer that carries a partial word in a register can publish complete u64s in order and never read the destination — the memset and the RMW tax both disappear. The one out-of-order region (the output chaining value, 256-bit aligned) is reserved and overwritten once the final state is known. BLAKE3 compresses each block with 56 rounds of its inner G function (the add-rotate-xor mixing step); this fixed 56-G schedule is fully unrolled by hand, with each round's state/message array indices written as fixed literals rather than computed at runtime, so the register allocator sees every round's actual data dependencies directly. Verified bit-identical to the OR-based builder, on real and padding slots, for both values of the prefix carry bit. Measured, paired: ST 87–90 → 64.8–68.9 ms (−24%, 3/3 disjoint); 8T −20%. Idea from the challenge tree; their further ≈3–7 ms SIMD-lockstep stage was priced against its ≈400 lines and not taken — the full network was later built anyway and measured null in-prove (§1.2).

1.2 The vectorized-packing port that became an allocation fix

Baseline: The lincheck stripe buffer was a fresh *zeroed* 128–512 MB allocation on every prove; its first-touch page faults dominated the witness bucket.

Improvement: Pool that one buffer (an untyped byte pool alongside the field-element pool) instead of reallocating and zeroing it each prove.

Mechanism: Not an operation-count change but an allocation fix: removes the per-prove first-touch page-fault cost of a fresh 128–512 MB zeroed allocation, at ≈40 lines (versus the ported vectorized-packing network's ≈200 lines, which measured null).

Result: In-prove witness bucket 18.5 → 8.0 ms at $n = 2^{16}$ and 74 → 33 ms at $n = 2^{18}$, both 2/2 paired at 8 threads.

The challenge tree's last witness edge was believed to be its lane-wise vectorized packing network: each bit-packed output stream owns a u32-granular writer whose pending word lives in a NEON lane, every wire bit is pushed by a single `vsli` (shift-left-insert) with compile-time shift constants, four compression blocks proceed in lockstep, and finished words dump contiguously through a `vld4` de-interleave. We reproduced the full design with a `build.rs` generator (≈200 committed lines emitting the unrolled kernel, rather than porting their 2.6k generated lines). It matched the existing witness builder's output bit-for-bit on its first full test run — and measured a *null* in-prove at both shapes.

The discrepancy exposed the real cost, which was never the witness builder: the lincheck stripe buffer — a repacking of z consumed by the lincheck — was a fresh *zeroed* 128–512 MB allocation on every prove, and its first-touch page faults dominated the bucket. Pooling that one buffer (an untyped byte pool alongside the field-element pool) took the in-prove witness bucket from 18.5 to 8.0 ms at $n = 2^{16}$ and 74 to 33 ms at $n = 2^{18}$, both 2/2 paired at 8 threads — more than the packing port's entire predicted value, at ≈40 lines. The mechanism is thread-count-agnostic (the buffer is faulted once per prove regardless); the 8-thread figures are simply where the campaign measured it. The quad kernel was reverted unmerged; its design survives in the commit history. This is the third time the campaign's largest available win sat in the allocation story rather than the kernel (the constant-region elision of §8.4, the pooled round-1 precompute, and this): audit where the buffers come from before porting compute.

2 Zerocheck round 1

Round 1 is the zerocheck's largest phase; the four kept changes below took it from 1477 to 1205 ms (−18.4%) at the benchmark shape.

2.1 Unreduced accumulation with multiplicative weight splitting

Baseline: Each y_K computed *reduced*: two `PMULL`s for the raw product plus a reduction costing four more, then a widening shift.

Improvement: Split the weight x^K multiplicatively so the reduction is deferred to the final fold; accumulate unreduced.

Mechanism: 6 `PMULL` → 2 `PMULL` per row-block (Algorithm 1).

Result: Round 1 1401.4 → 1273.4 ms (−9.1%), 8/8, every pair in $[-8.7\%, -10.1\%]$.

The prep kernel — round 1's per-row operand-weighting stage — computes, per K-row ($K \in \{0, \dots, 7\}$) and 16-lane block, $y_K = a_K \odot b_K \in \mathbb{F}_{2^8}$ and accumulates $\sum_K x^K y_K$ in 16-bit lanes, reducing once at the end. The baseline computed each y_K *reduced*: two `PMULL`s for the raw product plus a reduction costing four more, then a widening shift by K — six `PMULL`s per row-block for a reduction that the final fold makes redundant.

Let u_K be the *unreduced* 15-bit product. Then $\sum_K x^K u_K \equiv \sum_K x^K y_K \pmod{p_8}$, but $x^K u_K$ overflows 16 bits for $K \geq 2$. Split the weight so every factor lands where it is free:

$$x^K \;=\; \underbrace{(x^4)^{[K\geq 4]}}_{\text{choice of gather table}} \;\cdot\; \underbrace{(x^2)^{[\lfloor K/2\rfloor \text{ odd}]}}_{\text{byte-wise doubling of the reduced operand}} \;\cdot\; \underbrace{x^{K \bmod 2}}_{\text{in-lane shift}}\,.$$

The x^4 factor costs nothing per element: a second copy of the table with every byte pre-multiplied by x^4 scales each gathered XOR-sum by x^4, by linearity of T. The x^2 factor is a six-instruction shift-and-conditional-XOR on the already-reduced 8-bit operand (deg still ≤ 7). The residual shift is one vector instruction and raises term degree to at most 15.

Algorithm 1 Row-block accumulate, before and after

1: **before:** $acc \mathrel{\oplus}= \mathrm{shift}_K\big(\mathrm{reduce}(\mathrm{pmull}(da, db))\big)$ ▷ 6 `PMULL`
2: **after:** $da \leftarrow [K \geq 4]$? gathered from x^4 table : da; $da \leftarrow [(K/2) \bmod 2]$? $x^2{\cdot}da : da$
3: $acc \mathrel{\oplus}= \mathrm{shift}_{K \bmod 2}\big(\mathrm{pmull}(da, db)\big)$ ▷ 2 `PMULL`

Correctness needs the final reducer to be exact to degree 15, one beyond its documented "≤ 14"; both the scalar and vector reducers were verified exhaustively over all 2^{16} inputs (tests now permanent). Gather traffic is unchanged apart from the second 16 KiB copy of the table.

Measured: round 1 1401.4 → 1273.4 *ms (*−9.1%*),* 8/8*, every pair in* [−8.7%, −10.1%]. *The challenge tree's attribution predicted* 100–140 *ms; measured* 128.

2.2 The C-side message as a fold of the lincheck stripe

Baseline: The C-side banks, though a multilinear evaluation, were computed with the quadratic side's machinery: a bit transpose of every 8192-bit window plus 32 of the drain's 48 gathers per lane.

Improvement: Compute the banks as a true multilinear fold of the already-materialized lincheck stripe at the pinned challenges, undoing the over-folded eq factors with a closed-form constant.

Mechanism: Removes the per-window bit transpose entirely; the drain's gathers per (lane, b) drop from three to one.

Result: Round 1 1277.5 → 1204.7 ms (−5.7%), 8/8; added-fold cost ≈ 180 ms nets against ≈ 250 ms of removals.

In the fast hash setups $C = I$, so $\hat{c} = \hat{z}$ and everything round 1 emits on the C side is *linear* in the witness. Concretely the two banks are, for parity $p \in \{0,1\}$ and lane λ,

$$\mathrm{bank}_p[\lambda] \;=\; \sum_{K\equiv p\ (2)} \alpha^K \sum_b \frac{\gamma^b}{D} \sum_t \mathrm{eq}(r_{\geq 13}, t)\; z[\lambda, K, b, t], \tag{2}$$

a multilinear evaluation — which the baseline nevertheless computed with the quadratic side's machinery: a bit transpose of every 8192-bit window plus 32 of the drain's 48 gathers per lane.

By (1), a *true* multilinear fold at the actual pinned challenges reproduces the weights in (2) exactly. The prover already materializes the *lincheck stripe*, a repacking of z indexed $[\,i_{\mathrm{inner}} \,|\, i_{\mathrm{outer}}\,]$, for the later lincheck phase. So:

1: $v \leftarrow \text{fold\_stripe\_outer}(z_{\text{stripe}},\ \text{eq}(r_{k_{\log}..m}))$ ▷ the one $O(2^m)$ pass; same kernel lincheck uses
2: **for** $d = k_{\log} - 1$ **downto** 7 **do** ▷ data halves each round: $2^{14} \to 2^7$ elements
3: $\quad v[i] \leftarrow v[i] + r_d\,(v[i] + v[i + 2^d])$
4: **end for**
5: $\text{bank}_p[\lambda] \leftarrow \dfrac{\text{eq}(r_6, p)}{C_s}\, v[\,p \cdot 64 + \lambda\,]$ ▷ $C_s = (1{+}r_6)(1{+}r_7)(1{+}r_8)$, from r itself

Dimension 6 (the parity) and dimensions 0..5 (the lanes) are kept unfolded; the constant in step 3 undoes the two eq factors the partial fold applied, derived from the identity $Y_p = (C_s/\text{eq}(r_6, p))\,\text{bank}_p$ obtained by factoring (1) over the folded dimensions. With the banks gone from the drain, the workers run AB-only: the per-window bit transpose is deleted and the drain issues one gather per (lane, b) instead of three. Equivalence was proven at two levels before wiring: the new banks bit-identical to the drain's old output, and all three entry outputs (dense and padded) identical between the two implementations.

Measured: round 1 $1277.5 \to 1204.7$ *ms (*-5.7%*),* 8/8*. The cost of the added fold (*≈ 180 *ms) is included; the removals (*≈ 250 *ms) net out.*

2.3 Instruction-level parallelism in the gather drain

Baseline: One lane per iteration exposes three serial dependency chains (one per bank) to the out-of-order engine.

Improvement: Process two lanes per iteration so twice as many independent chains are in flight, with no change in total work.

Mechanism: 3 dependency chains in flight → 6 dependency chains in flight per iteration (same gather count and table size).

Result: Round 1 −63.1 ms (−4.3%), 8/8; combined with §2.4: 1477.3 → 1403.1 ms (−5.0%).

The drain accumulates, per output lane, three XOR chains of depth $n_{\text{med}} = 16$ (one per bank), each link a table gather feeding an XOR. The gathers are independent but the chains are serial, so one lane exposes three dependency chains to the out-of-order engine. Processing two lanes per iteration doubles that to six with no change in total work:

for $\ell = 0, 2, 4, \ldots, 62$ **do**
$\quad u_0, u_1, u_2 \leftarrow 0;\quad v_0, v_1, v_2 \leftarrow 0$ ▷ banks for lanes ℓ and $\ell{+}1$
$\quad$ **for** $b = 0 \ldots n_{\text{med}} - 1$ **do**
$\quad\quad u_j \mathrel{\oplus}= \text{convert}[b][\cdot];\quad v_j \mathrel{\oplus}= \text{convert}[b][\cdot]$ ▷ 6 independent chains in flight
$\quad$ **end for**
end for

This targets latency, not throughput: an attempt to shrink the gather *table* (64 → 8 KiB, doubling gather count) had cost 3.6% — the table already fit M1's 128 KiB L1D, establishing that the drain is bound by gather count and chain depth, not footprint.

Measured: round 1 -63.1 *ms (*-4.3%*),* 8/8*. Combined with Section 2.4:* $1477.3 \to 1403.1$ *ms (*-5.0%*).*

2.4 Structurally fixed rows of the b operand

Baseline: Every K-row of b is pushed through the $\mathbb{F}_2$-linear table T, including the 15 of 256 rows the circuit pins to 0^8 regardless of input.

Improvement: Guard each row with a single zero check; since $T(0) = 0$, a zero b-row contributes nothing and its work can be skipped outright.

Mechanism: Skips 64 table loads and 4 $\mathbb{F}_{2^8}$ multiplies for each of the 15 structurally-zero K-rows (out of 256).

Result: Round 1 -42.4 ms (-2.9%), 8/8.

Round 1's dominant kernel transforms the operands a, b through a table T implementing the inverse-NTT composition; T is $\mathbb{F}_2$-linear, so $T(0) = 0$. An exhaustive tally of the packed BLAKE3 witness (256 word positions per block $\times$ 256 blocks $\times$ 3 independent witnesses) shows the circuit pins 38 of 256 eight-byte K-rows of b regardless of the inputs, taking only three values:

$$\underbrace{\texttt{0xff}^8}_{\text{22 positions (const-one wires)}}, \qquad \underbrace{0^8}_{\text{15 positions (structural zeros)}}, \qquad \text{one mixed word.}$$

For a zero row the entire row's work vanishes: $db = T(0) = 0$, hence $y = da \odot db = 0$ contributes nothing to any accumulator. The kernel gains a single guard:

1: **if** $\mathrm{load}_{64}(b\_row) = 0$ **then return** ▷ skips all 64 table loads and 4 $\mathbb{F}_{2^8}$ multiplies

No tally data ships and no positions are tracked: the guard is exact for any witness, and a disagreeing witness falls through to the generic path. (The all-ones rows were also tried: constant-folding them saves only the b half of a row's work, and halving a row's loads halves its memory-level parallelism, returning 6 ms where 27 were predicted. Reverted; whole-row elimination is what pays.)

Measured: round 1 -42.4 *ms (*-2.9%*),* 8/8.

3 Zerocheck round 2

Two kept changes, sharing the deferred-reduction machinery, took round 2 from 433 to 312 ms (-28%).

3.1 Deferred reduction in vector registers

Baseline: The 256-bit accumulator lived as a four-word struct in general registers: six lane extracts per product to leave the vector file, four scalar XORs per accumulate.

Improvement: Keep the accumulator resident as two 128-bit vector registers and form the unreduced product with Karatsuba.

Mechanism: Karatsuba product: three `PMULL`s instead of the schoolbook four; accumulate: four scalar XORs $\to$ two vector XORs; the six lane extracts move from per-product to once per worker chunk.

Result: Round-2 phase 433.1 $\to$ 375.2 ms (-13.4%).

For $u \in \mathbb{F}_2^{256}$ (an unreduced carry-less product) let $R(u) \in \mathbb{F}_{2^{128}}$ be reduction mod $X^{128} + X^7 + X^2 + X + 1$. R is $\mathbb{F}_2$-linear, so for any products $u_i = a_i \odot b_i$ (unreduced),

$$\textstyle\sum_i R(u_i) \;=\; R(\sum_i u_i)\,,$$

and a sum of n field products needs one reduction, not n. The baseline already used this identity, but held the 256-bit accumulator as a four-word struct in *general* registers: each product paid six lane extracts to leave the vector file, and each accumulate was four scalar XORs.

The fix is representational, not algorithmic: keep the accumulator as two 128-bit vector registers. The unreduced product uses Karatsuba,

$$(a_l + a_h X^{64})(b_l + b_h X^{64}) : \quad ll = a_l b_l, \quad hh = a_h b_h, \quad \mathit{cross} = (a_l{+}a_h)(b_l{+}b_h) + ll + hh,$$

three `PMULL`s instead of the schoolbook four; accumulation is two vector XORs; the four extracts are paid once per worker chunk on the way out, and the existing scalar reducer is reused unchanged.

Measured: round-2 phase $433.1 \to 375.2$ *ms* *(*-13.4%*).*

3.2 The register-resident message loop

Baseline: The message loop crossed the vector/general register-file boundary three times per output pair (fold-accumulator extract, scalar field multiply in/out, re-entry for accumulation), at six `PMULL`s per pair.

Improvement: Keep the whole chain — fold, in-register Karatsuba multiply, and accumulate — resident in vector registers.

Mechanism: 6 `PMULL` $\to$ 5 `PMULL` per pair (three for the product, two for the fold through $X^{128} \equiv X^7{+}X^2{+}X{+}1$); all three per-pair register-file crossings removed, leaving only four stores and one eq load.

Result: Round-2 phase $358.1 \to 311.6$ ms (-13.0%), 8/8, every pair in $[-12.6\%, -13.3\%]$.

Round 2's message loop crossed the vector/general register-file boundary three times per output pair: the table-driven fold extracted its vector accumulator into a two-word struct; the message products $g_1 = a_1 b_1$, $g_\infty = (a_0{+}a_1)(b_0{+}b_1)$ went through the scalar field multiply (struct in, struct out); and the accumulate of Section 3.1 re-entered the vector file. The rewrite keeps the whole chain in vector registers: the fold returns its register unextracted; the products use an in-register Karatsuba multiply (five `PMULL`s — three for the product, two for the fold through $X^{128} \equiv X^7{+}X^2{+}X{+}1$ — against the baseline's six); the $\mathrm{eq} \cdot g$ products feed the Section 3.1 accumulator directly. Per pair, the only remaining memory traffic is the four required stores of folded values and one eq load.

Measured: round-2 phase $358.1 \to 311.6$ *ms (*-13.0%*), 8/8, every pair in* $[-12.6\%, -13.3\%]$*, taken on a machine with an active browser session (the min-of-5 protocol absorbs interactive bursts).*

4 Zerocheck rounds 3+

One structural change took the tail from 455 to 307 ms (-33%).

4.1 Fusing away a memory pass

Baseline: Two passes per worker chunk: fold and store a, b, then re-read the just-written multi-megabyte chunk from L2/DRAM to build the message.

Improvement: Fold each output pair in-register, store once, and consume the pair for the message while still in registers — no re-read.

Mechanism: Memory traversals per worker chunk: $2 \to 1$; `PMULL` count is unchanged from the two-pass form.

Result: Rounds-3+ phase $449.5 \to 306.6$ ms (-31.8%), 8/8, every pair in $[-31.1\%, -32.6\%]$ — the largest single win of the effort.

The tail rounds apply the register-resident treatment of §3.2 plus one thing round 2 had no opportunity for. The incumbent made *two* passes per worker chunk: fold $a_{\text{in}}, b_{\text{in}}$ into $a_{\text{out}}, b_{\text{out}}$, then re-read the just-written multi-megabyte chunk to build the message — an L2/DRAM read of data that had been in vector registers moments earlier. The fused kernel folds each output pair in-register ($e + r\,(e + o)$ via `mul_q`), issues the one required store, and consumes the pair for the message while still in registers:

for $x = 0 \dots lo - 1$ **do**
  $a_0, a_1, b_0, b_1 \leftarrow$ fold four input pairs at r ▷ in 128-bit NEON Q-registers
  store a_0, a_1, b_0, b_1 ▷ the required write, once
  $g_1 \leftarrow a_1 b_1;\quad g_\infty \leftarrow (a_0{+}a_1)(b_0{+}b_1)$ ▷ `mul_q`, never leaving registers
  $acc_1 \mathrel{\oplus}= \mathrm{eq}_x \odot g_1;\quad acc_\infty \mathrel{\oplus}= \mathrm{eq}_x \odot g_\infty$ ▷ unreduced, §3.1
end for

The `PMULL` count is identical to the two-pass form; the entire gain is one traversal instead of two plus the removed struct crossings. Notably, two earlier attempts on this exact loop had failed (a memory-interfaced pair-multiply kernel, +10%; wide accumulators alone, 0%): they changed the arithmetic encoding and the accumulator while leaving the pass structure intact, which is where the cost actually lived.

Measured: rounds-3+ phase 449.5 → 306.6 *ms (*−31.8%*),* 8/8*, every pair in* [−31.1%, −32.6%] *— the largest single win of the effort.*

4.2 The lookahead cascade: two rounds per pass

Baseline: After §4.1, each tail round is one fused pass (fold at ρ + next message): $\approx 2L$ read $+\,L$ write per round, halving L each time.

Improvement: Emit eight *lookahead* partial sums per pass, answer the next round's message without touching the arrays, and materialize two folds in one 4→1 sweep — round 2 hands the first lookahead to the tail, the exit hands the last fold to the boundary.

Mechanism: The post-fold message is a quadratic in the not-yet-drawn challenge whose coefficients are bilinear in pre-fold group values: eight per-group sums determine it for *any* ρ. The enabling arithmetic (from the challenge tree's kernel): all eight products of a group share its eq weight, so pre-scaling the four a-rows once makes each product a single unreduced widening multiply — 52 vs 72 PMULL per group. Intermediate half-size arrays are never written ($\approx 55\%$ of two sequential rounds' traffic).

Result: m=32 8T: tail 33–37 vs 53–66 ms (4/4 disjoint); round 2 pays +12 (its integrated kernel also emits the sums — the PMULL floor); zerocheck net −8–13 ms (sign 3/4). m=30: −1.0, no regression (2/2). Default on; `FLOCK_NO_ZC_LOOKAHEAD=1` reverts. Transcript byte-identical by test.

This reverses an early-campaign verdict: the same double-round idea lost 7–15% when first tried, and the regression was adjudicated to the technique. It belonged to the kernel — the scalar lookahead accumulator spilled what the NEON one-weight-per-group form keeps in registers. The technique and its implementation quality had to be judged separately, and only the cross-tree anatomy exchange (their kernel reached the same sums with 20 fewer PMULLs per group) exposed which was which.

4.3 Compact variant K: rounds 2–5 in two passes

Baseline: After §4.2, round 2 ran as two sweeps (fold + store the full pairs, then an L2 re-read for the eight lookahead products), and the tail bound the challenges from materialized 64-byte-per-pair state.

Improvement: The challenge tree's compact architecture, ported whole. Round 2's single sweep stores 48 bytes per pair — the folded even-row *anchor* plus the packed-byte XOR of the pair, still in the bit domain — and emits round 2's message AND round 3's as six deferred quadratic coefficients: the products are parity-split over pair index ($\mathrm{eq}_2(2y{+}1) = r_1\,\mathrm{eq}_3(y)$), so the odd halves ARE the round-3 aggregates after one r_1^{-1}. The consumer then binds ρ_1 and ρ_2 in one pass through two λ-scaled byte tables (bound = anchor + $\mathrm{fold}_\lambda(\delta)$ by fold linearity over $\mathbb{F}_2$; $\lambda_1 = \rho_1(1{+}\rho_2)$, $\lambda_3 = \rho_1\rho_2$), emits round 4's message, defers round 5's the same way, and materializes the quarter-size level where the cascade resumes unchanged.

Mechanism: Three stacked effects: one sweep instead of two (the parity split needs the same eight accumulators the lookahead used, but no re-read pass); −25% intermediate traffic in both directions; and challenge binding through table lookups instead of per-output multiplies. The degenerate-b fast path rides along: an all-ones packed b row folds to a per-table constant, so such pairs skip their 16 lookups and their provably-zero products — value-preserving, and common in this circuit's b operand.

Result: Producer 54.3–55.9 → 48.0–51.0 ms (3/3); tail 31.9 → 8.5 with the new ≈ 24 ms double fold between them; mechanism sum 85.9–87.9 → 79.9–84.5 (3/3 disjoint, avg −4.9) at m=32 8T grind-free. (An earlier parity claim compared against warm-window sub-line medians and is withdrawn: on bucket basis the zerocheck retains +14–19 ms at 8T after this change.) Transcript-identical by three tests including a targeted $b \equiv 1$ case.

4.4 Structured-b shortcuts and the hetero round-1 drain

Baseline: The round-1 prep kernel treated every 8-row block uniformly (its 16-bit geometric-eq machinery — shared with the challenge tree since the round-1 parity port — already skips zero b rows); the round-1 drain ran on the P pool alone.

Improvement: Two exact runtime shortcuts dispatched on two integer compares per block: an all-ones b block (the inv-NTT extension of the constant-one row is constant one, so $y_K = \mathrm{ntt}_a$) skips every b gather and every product multiply; a single-live-K_0 block (rows with $b = 0$ contribute nothing by $\mathbb{F}_2$-linearity) collapses to one dual transform and one lane multiply. Separately, the drain's fold/reduce pulls chunks from a shared counter drained by the rayon pool and two utility-QoS E-core threads.

Mechanism: Both shortcuts are value-preserving algebra, not approximation, and both patterns are common in this circuit's b operand; the challenge tree's further constants-census paths (positions hardcoded from a scored-distribution census) were left unported — the SHA-256 control had already adjudicated that family as noise. Random test data never exercises either path, so the oracle test crafts b with all-ones and single-K_0 quarters.

Result: Commit join wall (where the hoisted prep lives at 8T) 257.5–271.4 → 252.5–259.7 ms, 5/6 pairs, avg −5.7; drain hetero −0.53 ms 3/3 disjoint on its own line. Campaign-record run during certification: 454.70 ms / 576,521 comp/s.

5 Lincheck

5.1 One word load per stripe, two stripes per pass

Baseline: 16 loads per stripe step: 8 table entries plus 8 separate scalar byte loads for the indices.

Improvement: Load the 8 adjacent index bytes as one u64 with GPR shift-extracts, and fold two stripes per iteration so the table-entry XOR fuses to a single `EOR3`.

Mechanism: 16 loads/stripe $\to \approx 9$ loads/stripe.

Result: `partial_fold_z` ST 40.7–40.8 $\to$ 27.8–28.0 ms (-32%, 3/3 disjoint); 8T -28%.

The lincheck's dominant pass folds the 128 MB packed witness through per-stripe 256-entry byte tables into eight Q-register accumulators. The loop is load-port bound (M1: 3 loads/cycle), and the incumbent spent 16 loads per stripe step: 8 table entries *plus 8 scalar byte loads* for the indices. The indices are 8 adjacent bytes — one u64 load and GPR shift-extracts replace all eight — and folding two stripes per iteration lets the two table entries enter the accumulator as an XOR pair, which LLVM fuses to a single three-input XOR instruction, `EOR3`, available under ARM's SHA3 feature: ≈ 9 loads per stripe, same XOR multiset, output bit-identical to the baseline. Measured, paired: `partial_fold_z` ST 40.7–40.8 $\to$ 27.8–28.0 ms (-32%, 3/3 disjoint) — landing on the pass's computed ≈ 28 ms load floor; 8T -28%. This supersedes the earlier "no reachable headroom" verdict, which had priced blocking and table-geometry changes but not the index-load restructure (idea from the challenge tree's asm kernel, re-expressed in intrinsics).

6 PCS open

6.1 Composing the block scalar into the fold table

Baseline: The combine pass spent one field multiply plus one 16-load table fold per slot per claim — 58% of open time.

Improvement: Fold the block-constant multiply into a freshly composed per-(claim, block) table, built once from 127 shift-and-fold doublings plus subset-sum expansion, so the sweep only indexes the composed table.

Mechanism: Removes $2L \approx 16.8$M per-slot field multiplies per open, replaced by a one-time ≈ 4.3k-word-op build per block (127 shift-and-fold doublings plus $16 \cdot 255$ subset-sum additions).

Result: Combine 94.4 $\to$ 78.5 ms, open total $\approx 160 \to\approx 145$ ms (-9%); at 8 threads combine is 10.9 ms, a near-linear transfer.

Shape for this section: the 2^{16}-compression open ($m = 30$, $L = 2^{23}$ packed slots, two batched claims). The ring-switch reduction leaves, for each claim k, an eq tensor in factored form $\mathrm{eq} = \mathrm{eq}_{\mathrm{lo}} \otimes \mathrm{eq}_{\mathrm{hi}}$ (lengths B and L/B) and an $\mathbb{F}_2$-linear map $L_k : \mathbb{F}_{2^{128}} \to \mathbb{F}_{2^{128}}$ (the γ_k-weighted byte-table fold). The *combine* pass materializes the batched basis

$$b[j] \;=\; \sum_k L_k\big(\mathrm{eq}_{\mathrm{lo}}[j \bmod B] \cdot \mathrm{eq}_{\mathrm{hi}}[\lfloor j/B \rfloor]\big), \qquad j < L,$$

and was the largest bucket of the open: one field multiply plus one 16-load table fold per slot per claim, 58% of open time.

The multiply is removable. Multiplication by the block constant $e = \mathrm{eq}_{\mathrm{hi}}[\lfloor j/B \rfloor]$ is itself $\mathbb{F}_2$-linear, so the per-slot map $G_{k,e}(w) = L_k(w \cdot e)$ is a composition of $\mathbb{F}_2$-linear maps and is fully determined by its 128 monomial images $g_r = L_k(X^r e)$. Once per (claim, block), encode $G_{k,e}$ as a fresh 16×256 byte table: advance $X^r e$ by 127 exact shift-and-fold doublings (multiplication by X costs one shift and a conditional XOR of the reduction word), evaluate L_k on each via the existing table, and expand every byte position from its 8 generators by subset-sum doubling

($16 \cdot 255$ additions). About 4.3k word operations per block; the sweep then indexes only the composed table, and the per-slot multiply — $2L \approx 16.8$M multiplies per open — is gone.

The catch is the block length. The balanced split used by the many-pass folds ($B = 2^{\lceil n/2 \rceil} = 2^{11}$ here) is too fine for the build to amortize; the deferred claims carry their own coarse split $B = \min(2^{15}, 2^{n-8})$, making $\mathrm{eq}_{\mathrm{lo}}$ a single 512 KiB L2-resident stream shared by every block and the build ≈ 0.4 ops/slot. Correctness is exact, not approximate: every composed entry is an XOR combination of exact field products — the same XOR multiset as the slot-multiply path — so the transcript is bit-identical to the slot-multiply path's (unit equivalence test; verification of full proofs unchanged).

Measured (ST, minimum over samples): combine $94.4 \to 78.5$ ms, open total $\approx 160 \to\approx 145$ ms (-9%); at 8 threads the combine is 10.9 ms, a near-linear transfer. The idea was found dormant in the challenge tree (present, never isolated or measured there) and re-derived.

Rejected alternatives for the open. (i) Three-way XOR fusion (`EOR3`) in the fold's reduction tree is flat: LLVM already emits it under `target-cpu=native`, and the sweep is bound by its 32 loads per slot, not its XOR count. (ii) Fusing both claims into one sweep — two live composed tables, one store, no read-back of the intermediate — *regresses* 15%: 128 KiB of gather targets thrash L1, so the claim-sequential order stands. (iii) The round-1 lookahead coefficients accumulated in the combine's tail are an accounting wash, not a speedup: tail-plus-initial-sumcheck costs 60.5 vs. 60.6 ms across the two trees' opposite placements of the same work.

6.2 The direct open: sufficient statistics instead of a basis

Baseline: Each claim is materialized as a length-L basis vector; the combine builds $b = \sum_k \gamma_k B_k$ and the sumcheck folds it beside the witness — the basis' whole life is $\approx 4L$ element-ops plus the matching DRAM traffic, and it is what made the open scale linearly in L.

Improvement: Never materialize the basis. Each claim carries a 128×2^c banked statistic, captured for free where the witness already streams; the first c sumcheck rounds run on the banks, and only the $L/2^c$ residual basis is ever built.

Mechanism: B_k is a rank-1 eq tensor pushed through an $\mathbb{F}_2$-linear map — the sumcheck messages it feeds are computable from a constant-size contraction, so the $O(L)$ representation was pure convenience.

Result: At $m = 32$: open $97.4 \to 64.1$ ms 8T (3/3 disjoint) and $614 \to 210$ ms ST; whole proof 527–543 $\to$ 484.3–518 ms, best 484.25 ms (541k c/s). Proof bytes identical; `FLOCK_NO_OPEN_DIRECT=1` restores the basis path.

Shape for this subsection: the ranked 2^{18}-compression open ($m = 32$, $L = 2^{25}$, two batched claims). After ring-switching, claim k is the inner product $\langle f, B_k\rangle = t_k$ against $B_k[i] = \varphi_k\Big(\gamma_k \prod_j \mathrm{eq}(u_j^{(k)}, i_j)\Big)$ — a rank-1 eq tensor at the claim point pushed elementwise through the $\mathbb{F}_2$-linear ring-switch map $\varphi_k(u) = \sum_b \mathrm{bit}_b(u)\, E_k[b]$. The incumbent evaluates this rank-1 object by writing out all 2^ℓ of its entries and then folding them c times.

Split the word index $i = (e, h)$ into its c lowest and $\ell - c$ highest coordinates, so the tensor factors as $\mathrm{lo}_k(e) \cdot \mathrm{hi}_k(h)$, and define the *banked statistic*

$$M_e^{(k)}[\beta] \;=\; \sum_h \mathrm{bit}_\beta\big(f[(e,h)]\big)\, \mathrm{hi}_k(h) \qquad (2^c \text{ banks of 128 slices}),$$

which is the ordinary $\hat{s}_v$ contraction with the low c coordinates left unfolded — and which the prover computes *anyway*: the AB claim's banks fall out of lincheck's pre-sumcheck z-fold, the C claim's out of zerocheck round 1's stripe fold, in both cases by stopping an existing small fold c coordinates early. Two states per claim then carry the open: the per-bank transpose $A[b, e]$ (bits extracted *before* any $\mathbb{F}_{2^{128}}$ binding — the step that makes folding sound) and $W[b, e] = \varphi_k(\gamma_k \operatorname{lo}_k(e) \cdot x^b)$. Both fold bank-wise at the sumcheck challenges, each round's message is $\sum_e \langle A(x_0, e), W(x_0, e)\rangle$ over the 128-slice axis — $O(2^{c-r})$ inner products, no $O(L)$ touch — and after the c lane folds W's surviving bank *is* the byte-map generator from which the $L/2^c$ residual basis is materialized and the incumbent recursion resumes. With c equal to the L0 fold count, the banked region spans exactly the rounds where the basis was large. Every quantity is the same field element the incumbent computes, reassociated — so the proof is byte-identical (test-pinned), and the kill switch restores the basis path exactly.

Measured at the ranked shape (paired, alternating, same binary): open 94.3–100.2 → 59.5–72.1 ms at 8 threads (3/3 disjoint) and 614 → 210 ms single-threaded; zerocheck unregressed (the banked captures ride existing folds); whole-proof best 484.25 ms / 541k compressions/s — the campaign's best figure at this shape. This is the table-vs-fold rule of the Preliminaries taken to its endpoint: the claim is one resident rank-1 object, so contract once and never materialize the map. Idea from the challenge tree's ranked open (located from its honest per-phase numbers after an instrumentation artifact had hidden it); algebra re-derived, implemented as four independently tested units.

6.3 Refinements to the direct open

Baseline: The freshly ported direct open spent 8.7 ms building its per-claim states (a 128-bit conditional scan per W entry, serial per-bank transposes) and 26.4 ms on the L0 lane folds (one halving pass of f per round, $\approx 2L$ traffic).

Improvement: Build W through the standard 16-lookup fold byte table; parallelize the bank transposes; and fold f ONCE — all k lane challenges composed into a single $2^k \to 1$ pass.

Mechanism: The composed fold is an option only the direct open has: its round messages come from the banked states and never touch f, so the witness is needed only at the level-1 boundary — $\operatorname{out}[j] = \sum_{e<2^k} \operatorname{lag}[e]\, f[2^k j + e]$ with $\operatorname{lag} = \operatorname{build\_eq}(\rho_0..\rho_{k-1})$, the same field elements as k successive folds at $\approx 1.03L$ traffic instead of $\approx 2L$.

Result: Open 59.7–61.3 → 45.5–50.4 ms at 8T (−20%, 3/3 disjoint); whole proof 516–538 → 492.5–511 ms. State construction 8.7 → 1.4 ms (the challenge tree's floor for the same stage is 0.5). Proof bytes identical throughout.

Three local costs of the port, removed without touching its algebra. The W-state entries $\varphi(\gamma \operatorname{lo}(e)\, x^b)$ are exactly `fold_b128_elems`' per-element map, so the existing byte-table build applies verbatim; the per-bank 128×128 bit transposes are independent and parallelize; and — the structural one — because the direct messages are computed from the banked states alone, nothing needs the partially folded witness until the residual basis is materialized, so the per-round fold chain collapses into one composed Lagrange pass. The dense path could never do this: its every round message reads the folded pair. Certified by a two-binary paired A/B (the three fixes share one keep decision), transcript identity re-checked at each step.

6.4 Truncating the induce transpose at rate $1/2$

Baseline: The sparse-prefix induce scatter-transposes the query indicator over the full 2^{n+1} codeword domain, then `truncate`s to the low 2^n coefficients — the final transpose layers compute a half that is thrown away.

Improvement: Fuse the final three transpose layers into one strided kernel that computes only the retained low half (idea from the challenge tree; exact precisely when $\log(1/\text{rate}) = 1$).

Mechanism: The transpose runs forward layers in reverse order, so the last three pair at distances $n/8$, $n/4$, $n/2$. An in-place 8-gather kernel applies the three butterfly levels in registers and writes the four low outputs; the final layer's retained output is the plain sum $a + b$, so its multiplies vanish entirely. Three full sweeps ($\approx 6n$ traffic) become one n-read/$\frac{n}{2}$-write pass.

Result: Induce 6.8–7.6 → 6.0–7.2 ms at m=32 8T (trace attribution); certified jointly with §6.5 (open bucket 40.8–41.3 → 36.8–37.8 ms 8T, −9%, grind-free protocol). Retained-half byte equality with the dense transpose by test.

6.5 Lazy out-of-domain binding

Baseline: Each OOD sample z builds the dense $\text{eq}(z, \cdot)$ table, runs the fused message+eval pass over (f, eq_z), and combines $\alpha \cdot \text{eq}_z$ into the running basis — $\approx 7L$ traffic per sample, five samples at L0 alone.

Improvement: Never materialize eq_z: factorize it as $\text{eq}_{\text{lo}} \otimes \text{eq}_{\text{hi}}$, read only f for the message, and defer every sample's basis combine into the level's basis glue, drained fused in its single pass (idea from the challenge tree).

Mechanism: The eq table is LSB-first, so splitting z splits the table as a tensor: $\text{eq}_z[x] = \text{eq}_{\text{lo}}[x \bmod 2^k] \cdot \text{eq}_{\text{hi}}[\lfloor x/2^k \rfloor]$. The hi factor is constant per 2^k-block, so block partials against the L1-resident lo table are scaled once per block — the same multiply count per pair as the dense kernel with no table build or re-read. Glue stashes $(\alpha \cdot \text{eq}_{\text{hi}}, \text{eq}_{\text{lo}})$; the next basis glue adds all pending tensors alongside $\beta \cdot b_{\text{new}}$ in its one read-modify-write (fold paths flush as insurance). Bit-identical by distributivity and order-free $\mathbb{F}_{2^{128}}$ sums.

Result: OOD stage 2.6–3.0 → 0.4–1.0 ms at m=32 8T; the draining basis glue pays +0.1–0.6. Certified jointly with §6.4.

6.6 Fusing the boundary pass

Baseline: At the direct open's level-1 boundary, the composed witness fold (one streaming pass over the full witness, DRAM-bandwidth-bound) and the residual-basis build (byte-table folds, 16 L1 gathers per element) ran back-to-back: $4.5 + 5.6$ ms.

Improvement: Two stages. First, `rayon::join` — they are data-independent and stress different resources, so they share the pool (10.1 → 7.3–8.4). Then full fusion (from the challenge tree's materialize pass): one sweep over the witness produces *both* arrays — the gathers and the factorized eq products execute in the witness stream's stall slots.

Mechanism: Pool-level overlap recovers only part of the idle issue slots; instruction-level fusion recovers the rest. Per output: the $2^k \to 1$ Lagrange fold accumulates unreduced (3 vs 6 PMULL per product, one reduction), and $b_1[j] = \sum_b \text{fold}_b(\text{eq}_{\text{lo}}[j \bmod B] \cdot \text{eq}_{\text{hi}}[j/B])$ — the suffix eq tensor is never materialized. The standalone split-tensor fold was adjudicated *slower* (its per-slot multiply outweighed the saved tensor build as its own pass); fused under the stream, that verdict inverts. Exact: deferred reduction is linear over XOR, field multiplies are exact, sums are order-free.

Result: Boundary pair 10.1 → 5.5–5.7 ms; the open's initial-sumcheck stage 11.3 → 5.9–6.0 ms at m=32 8T (challenge tree: 5.19). Join stage certified with §6.7 (open sign 3/3); fusion stage phase-paired 3/3 disjoint (5.85/6.04/6.59 vs 7.52/8.40/8.13) — the bucket-level A/B could not resolve the ≈ 2 ms effect in that window's noise, the per-phase pairing could.

6.7 Cache-blocking the induce transpose

Baseline: The sparse-prefix induce's dense remainder ran one full-array parallel sweep per transpose layer — ten sweeps over the 2^{21}-element domain at m=32 L0, ≈ 640 MB of traffic.

Improvement: Run every layer whose blocks fit an L2-resident chunk in a single parallel pass over chunks, layers applied back-to-back in cache.

Mechanism: The transpose processes high layers (small blocks) first, and blocks nest: a chunk sized to the run's lowest layer contains complete blocks of every higher layer, so a 2 MB chunk hosts nine consecutive layers with one read and one write of DRAM traffic (≈ 130 MB total) — the same sub-group decomposition the interleaved encode NTT already uses for its deep layers, applied to the transpose direction.

Result: Induce 6.6–8.5 → 5.4–6.7 ms at m=32 8T. Certified jointly with §6.6: open bucket paired sign 3/3 ($-2.8/-3.2/-8.0$, avg -4.7 ms), grind-free protocol. Byte-equality tests cover the blocked run with and without the fused low-half tail.

6.8 Recursive commits from the message

Baseline: Each recursive Ligerito level's commit replicated its folded witness into the codeword buffer (a full write plus read-for-ownership), then transformed from the rate layers: L1 alone is a 32 MB fill ahead of the NTT.

Improvement: The first fused top pass reads its rows straight from the compact folded witness (`from_message`), so the standalone replicate pass and its RFO traffic disappear.

Mechanism: The same fused pass had been built for the *main* commit and adjudicated null there (its join window is compute-limited, so deleted traffic bought nothing); the recursive commits run standalone and bandwidth-bound, where the challenge tree's equivalent measures fill = 0.00. The reverted patch was resurrected for this path only.

Result: L1 fill 0.5 → 0, all-in NTT 5.2–6.9 → 5.0–5.2; recursive commits 14.3–15.5 → 12.9–13.3 ms at m=32 8T. Open bucket paired sign 3/3 ($-2.35/-1.60/-0.56$, avg -1.5), totals 3/3.

7 Merkle hashing: BLAKE3

7.1 Two transposed four-wide states in flight

Baseline: The crate's NEON backend runs a single 4-wide transposed state (one `uint32x4` per state word); BLAKE3's serial G function leaves Apple's four NEON pipes half idle.

Improvement: Interleave a second independent transposed 4-wide state, G-for-G, to fill the idle pipe slots.

Mechanism: 1 transposed 4-wide state (4 messages in lockstep) → 2 interleaved 4-wide states (8 messages in flight, the full 32-register NEON file); no added work per message.

Result: ST tree build 1.63 → 2.30 GB/s at 512 B leaves (+41%) and 1.71 → 2.42 GB/s at 1 KiB leaves (+42%).

The PCS Merkle tree can hash with SHA-256 (hardware silicon, ≈2.5 GB/s per core) or BLAKE3. The blake3 crate's NEON backend processes four messages in lockstep — one `uint32x4` per state word — but a single 4-wide state is *latency*-bound: BLAKE3's G function is a serial add–xor–rotate chain, and Apple's four NEON pipes sit half idle waiting on it. Interleaving a *second* independent transposed 4-wide state, G-for-G (eight messages in flight, the full 32-register NEON file), fills the pipes — the same independent-chains pattern as §2.3. Dispatch: groups of eight leaves or parent pairs take the new kernel; tails fall to the crate path; byte-identical to the crate's original kernel by an equivalence test over all batched message sizes, counts, and flag combinations.

Measured (ST tree build): $1.63 \to 2.30$ GB/s at 512 B leaves (+41%, now 1.08× *faster* than SHA-256) and $1.71 \to 2.42$ GB/s at the 1 KiB production-geometry leaves (+42%, parity with the SHA silicon). The challenge tree reaches ≈2.58 GB/s with a 2.6k-line generated assembly kernel; this is ≈290 lines of intrinsics within 6% of it — LLVM absorbed the full register pressure without measurable spill cost. End-to-end the change erases the −5% penalty BLAKE3-Merkle previously carried in this tree, making the Merkle hash choice free.

8 Multithreading

The campaign target mirrors the single-threaded closure: CPU-only multithreaded throughput within 10% of the challenge tree. Start: 1.19× apart at the 2^{16} shape. Standing after the two subsections below: 1.146× (149.3 vs 130.2 ms, paired same-minute, production configuration). All numbers here are 8-thread-class production runs at 2^{16} compressions, on Apple silicon's split between performance cores (P-cores) and efficiency cores (E-cores). Several results below are a paired *kill-switch A/B*: the optimization is gated behind a runtime toggle, so disabling it reproduces the pre-change baseline in the same binary.

8.1 Round-1 AB prep under the commit, on all cores

Baseline: Overlapping the witness-only AB transform with the commit under `rayon::join` measured a wash twice on the P-core-only pool; the transform's output buffer was a fresh zeroed allocation.

Improvement: Draw the output buffer from the scratch pool uninitialized, and run the join window on the all-core (P+E) pool instead of P-cores only.

Mechanism: Not an operation-count change but a memory-traffic and scheduling fix: removes the memset-plus-page-fault cost of a fresh zeroed allocation, and moves the overlap window onto the otherwise-idle E-core issue capacity left by the bandwidth-bound commit NTT.

Result: Kill-switch A/B 3/3 at 2^{16} (149.6–151.3 vs 152.5–156.3 ms), 2/2 at the ranked shape (clean pair −57 ms).

Round 1's dominant half — the shift-reduce AB transform — reads only the witness, so it can run before the commitment root exists. Overlapping it with the commit under `rayon::join` was measured a wash TWICE on the 8-P-core pool: both arms time-slice a saturated pool. Two changes flip it decisive: (i) the transform's 2^{m-13} KiB output buffer comes from the scratch pool *uninitialized* (a fresh zeroed allocation's memset + page faults consumed the entire overlap gain; slots for padding-skipped rows stay stale-but-unread, bounded by the same padding table every consumer uses), and (ii) the join window runs on an all-core (P+E) pool while the rest of the prove stays on P-cores. The efficiency cores are the active ingredient: the commit's NTT is bandwidth-bound and leaves issue capacity, and the prep is gather/PMULL compute the E-cluster can add without competing for DRAM. (The converse placement — a bandwidth-bound task like the lincheck stripe transpose on the E-cores during the commit — measured *negative*.) Paired kill-switch A/B: 3/3 at 2^{16} (149.6–151.3 vs 152.5–156.3 ms), 2/2 at the ranked shape (clean pair −57 ms). Bit-identical: the precomputed and inline transforms are equality-tested on all outputs.

8.2 The streaming commit: pipelined leaves and the fill's overlap budget

Baseline: The commit ran as stages — replicate-fill, NTT, Merkle — joined with the round-1 AB prep on the all-core pool. The window was compute-additive: the prep overlapped only the short fill stage, then $\approx$ 180 ms of transform and hashing ran with the prep already finished ($\approx$ 273 ms wall).

Improvement: The challenge tree's architecture, ported whole: the deep NTT pass publishes each finished sub-group as $\approx$1-MiB leaf jobs into a small bounded queue drained by utility-QoS helper threads (which the scheduler steers to the efficiency cores); each job hashes its leaves *and* its aligned local parent subtree while the lines are hot; a full queue hashes inline on the publisher; the tail drains on the main pool; only the tiny shared top levels remain after (0.03 ms). Crucially, the fill is fused into the first top pass (from-message), so the prep overlaps the *entire* encode+leaves window.

Mechanism: The fill fusion is the load-bearing piece, not the pipeline: without it the prep burns its overlap budget beside a standalone bandwidth stage that needs none of it (the v1 port measured *worse* than staged). The staged-shape adjudication of fill fusion — "deletes a read, not compute", null — does not transfer: in the pipeline shape the deleted stage is what buys the prep its hiding window. Null verdicts are architecture-scoped.

Result: Join wall (= commit + prep, one window) 260.5–262.6 $\to$ 252.5–254.7 ms (3/3 disjoint); whole proof 479.8–502.8 $\to$ 456.9–482.5 ms (3/3 disjoint, avg −23). Campaign record 456.95 ms / 573,681 comp/s at m=32, grind-free. Tree and root bit-identical by test.

8.3 The ranked radix-8 top: dual from-message pass, staged stores, hetero tiles

Baseline: After §8.2, the commit's top layers ran as a from-message fused-2 pass plus three fused-2 sweeps, all on the P pool (the E-core helpers idle until the deep pass publishes leaf jobs).

Improvement: Layers 1–9 as *three* radix-8 passes: layer 1 is a dual-destination from-message kernel (one witness read produces both replica blocks; block 0's spine twiddles are zero, so its chain is XOR-only), layers 4 and 7 are in-place sweeps with q-register-resident butterflies; every pass distributes 128-row tiles across the rayon pool AND two utility-QoS E-core threads via one shared work counter.

Mechanism: Radix-8 was not the earlier fused-3 failure — the kernel shape was. Fresh-destination scatter stores (sixteen 16-B streams, tens of MB apart) defeat the streaming-store detector and pay a read-for-ownership on the whole 1-GB destination; staging each row group in an 8-KB L1 tile and emitting sequential non-temporal bursts restores fill-like store behavior. In-place passes need no staging (their lines are already owned). One fewer full sweep of traffic, one witness read instead of two, and the E-cores now assist the top before switching to leaf hashing.

Result: Commit join wall 273.1–275.5 $\to$ 256.6–266.7 ms (3/3 disjoint, avg −13.1; commit bucket 3/3). Bit-identical by the from-message and commit equality tests.

8.4 Witness constant-region elision via scratch provenance

Benedikt: what is this title? Use plain english please

Baseline: The full-write witness builder rewrites every byte of z/a/b each prove, including regions — b's all-ones prefix, reserved-slot words, all three streams' zero tails — identical across every prove of a layout.

Improvement: Tag a pooled buffer with its provenance — which layout's output it currently holds, derived independently at give and take sites — and, on a tagged hit, skip the constant-region writes.

Mechanism: No explicit byte or operation count is given in this section; this is a work-removal via provenance tracking, not a stated op-count reduction.

Result: Paired kill-switch A/B: 4/5 (147.0–149.0 vs 148.3–152.7 ms).

The full-write witness builder rewrites every byte of z/a/b each prove — including b's all-ones prefix and reserved-slot words and all three streams' zero tails, which are identical for every prove of a layout. A provenance tag records which prove-layout's output a pooled buffer currently holds. It is derived independently at the buffer's give site (from the R1CS constants) and at its take site (from the encoder's own constants) — no plumbing carries it — so a match marks the buffer as holding exactly a previous prove's output of this layout; any other pool custody clears the tag. On a tagged hit the builder skips the constant-region writes. Idea from the challenge tree's scratch tokens, re-derived; paired kill-switch A/B: 4/5 (147.0–149.0 vs 148.3–152.7 ms). Byte-identity is enforced by a test that compares a tagged give/take cycle's output against a fresh, non-skipped generation.

Rejected alternatives for multithreading. Seven mechanisms measured null or inverted on this machine, all paired: the P-pool-only join (twice); non-temporal stores in the witness writers (M1 ignores the hint — third confirmation); the stripe transpose on E-cores during the commit (−, bandwidth-on-bandwidth); the all-core PCS combine; four-layer NTT fusion on aarch64 (+19–26%, sixteen live F128s spill); a two-block scalar witness interleave (+40% ST, GPR blowout); a 4-wide SIMD witness builder with scalar packing (the packing, not the hash math, is the cost); and the full lane-wise vectorized packing network itself (§1.2 — null once the stripe-buffer allocation was fixed). What separates the remaining few percent at the production shape: the challenge tree's compact round-2 anchor+delta format with symbolic round-collapsing lookaheads — a thousand-line-class port, priced and declined at the current bloat bar (the campaign's implicit line-count-vs-payoff threshold) pending an explicit call.

9 After the main merge: the union protocol

On 2026-08-28 the branch absorbed the repository's main line, and with it a different statement: the *union* prover (one witness for a registry of circuit types, batch-major, count-proportional lincheck, an assist that binds the jagged layout), the $\mathbb{F}_{2^{256}}$ two-point out-of-domain split in the Ligerito open with per-level OOD binding and five recursive levels, and a second zerocheck — the AG (genus-95) univariate skip — beside the RS one. Every entry above was re-audited against that tree before anything new was built. Live already: §7.1, §5.1, the round-1 kernel work (§2.1, §2.3, §2.4) and the round-1 E-core drain. Dead by protocol: §6.2 and §6.3 (the $\mathbb{F}_{2^{256}}$ split replaced the basis open), §2.2 (the union is batch-major), and every open entry that targeted the retired basis-combine and sparse-prefix induce machinery. Measured against main at production settings, the surviving wins were worth 1.027× (8/8) — nearly all of it §5.1. What follows is the work on the new protocol, in pipeline order as before. The benchmark shape throughout is m=32 (a 2^{18}-compression BLAKE3 union, 2^{25}-word witness), grind-free, on the same M1 Max (8P+2E); "MT" is the eight-thread P-core pool.

9.1 Round 2 on the union: the deferred-reduction port

Baseline: Main's shared round-2 kernel accumulated the 256-bit message partials in the four-word general-register struct that §3.1 had replaced; the x86 arm already accumulated wide, only aarch64 had been left behind. Every register-resident primitive of §3.2 was in the tree, reachable only from the dormant compact-K block.

Improvement: Route both round-2 flows (sparse and dense) through the vector-resident accumulator, then the whole fold → multiply → accumulate chain through the q-register primitives.

Mechanism: As in §3.1 and §3.2; bit-identical (the XOR multiset is unchanged, reduction is $\mathbb{F}_2$-linear), all thirteen proof-byte pins hold.

Result: Round 2, ST: 281.7 → 239.3 ms (−9.2%, 3/3), then 267.2 → 236.1 (−11.6%, 3/3); −16.2% cumulative, on a baseline already cheaper than the old tree's because the union's sparse path folds only the live 72% of the domain.

The compact-K rounds and the cascade tail (§4.3, §4.2) were wired onto the union's grinding driver in the same pass, with the two test oracles and the transcript-identity tests restored. They did *not* keep the bench: the union's boolean region is 72% occupied, so main's support-proportional sparse route fires first, and compact-K's producer must pay a full-domain pass to build its tables — 145–190 ms for rounds 2–5 plus tail against the sparse route's 93–170. The kernels are cheaper (K fold 31–48 and cascade tail 11–15 against the sparse tail's 44–103); the producer eats the difference. The cascade now serves dense single-run flows, where the sparse gate fails and the 2026-08-27 certification applies; the sparse route dispatches first everywhere else. A same-binary route A/B settled the question in both directions at once: sparse round 2 43.4 against dense 101.7 (8/8), but the cascade tail 10.6 against the sparse tail 44.9 (8/8) — the two halves were disjoint in opposite directions, and the win was a sparse four-to-one lookahead kernel that neither tree had (§9.5).

9.2 The AG skip fold: a scalar twin of an existing kernel

Baseline: The AG path's skip→multilinear fold folded each 64-bit message word through $8 \times [\mathbb{F}_{2^{128}}; 256]$ tables with a scalar byte-dot: eight table loads and eight struct XORs per word. Probes put 85% of the fold in these gathers and 12% in the message chain, so the round-2 treatment (§9.1) does not apply — the accumulator is not where the time is.

Improvement: The RS row fold `fold_one_row_neon_q` has the identical table layout (8×256 entries, contiguous, 4 KB stride); it is re-exported and the AG byte-dot routed through it on aarch64.

Mechanism: Same table entries, same XOR multiset — bit-identical — with the eight gathers issued as vector loads and XORed in the vector file.

Result: AG fold −3.1 ms MT (−7.5%, 3/3), −28 ms ST (−9.0%, 2/2); the first improvement to the AG path.

Smaller than a first look at the scalar source suggested, because LLVM already emitted reasonable code for the 16-byte-aligned table loads; what the kernel recovers is the remaining gap. The lesson generalizes: when the same fold appears in two protocols, the kernel written for one is usually a drop-in for the other, and the audit that finds it is cheaper than a new kernel.

9.3 The AG zerocheck as the default: dispatch, not kernels

Baseline: The flat prover's default zerocheck was RS; AG existed as a parallel entry with its own prove→verify test, exercised only by the tower and a micro-bench. Its sparse route had been (mis)measured as its multi-thread pessimum.

Improvement: AG is the default on aarch64 (RS stays the default elsewhere, and is retained as the x86 fallback: the AG round-1 kernel is NEON-only and the CI's x86 leg proves with RS); the SHA-256 setup gets the mechanical twin of the BLAKE3 AG entry; the AG proof gets its own wire flavor so the bench reports peak memory, verify and proof size on both arms.

Mechanism: Two corrections came first. Sparse is AG's optimum on *every* stage (round 1 55.7 vs 57.1, fold 41.1 vs 47.2, tail 47.8 vs 65.7, zerocheck+lincheck 183 vs 209 ms; end to end 4/4) — the earlier "3.2×" reading was a timing artifact — so the dense/sparse gate follows the flavor. Then the promotion measured in the shipped configuration, both arms on the default sparse gate.

Result: Prove median 966.1 → 894.8 ms (−8.2%, 4/4; deltas $-112/-84/-75/-68$), best 935.0 → 851.4: worth more than every kept port of the campaign combined (1.027×), and it is not new kernel code.

The RS path cannot be deleted — it owns every transcript-byte pin and x86 proving — so both entries stay, as the recursion plan prescribes. The subsequent round-1 audit against AG found nothing above the bloat bar: the multiplicative-weight split of §2.1 has no reduction to defer in AG (its products are bitsliced $\mathbb{F}_2$ ANDs, and its eq-weighting already accumulates unreduced), structured-b of §4.4 has a 2% ceiling there, and the stripe-C derivation of §2.2 maps onto batch-major addresses (skip = in-word bits 0–5, parity = bit 6, inner = batch bits 0–5, outer = the rest) but costs more multiplies than it removes transposes: AG's C side is a multiply roofline and its transpose is free.

9.4 Witness generation: attributed, and the dead regions skipped

Baseline: The trace printed 0.00 ms for witness generation because the timer wrapped a passthrough; the real generator ran outside the traced region, 64 ms (7.6% of the prove) that no session had attributed. Attributed: 1.5 GB of non-temporal stores in 25.7 ms (a 58 GB/s "roofline"), 17.8 of BLAKE3 compute, a 432 MB padding memset (15.0), the lincheck stripe (12.8) and its 153 MB tail clear (7.8), row fill (6.2).

Improvement: Three changes. (i) The a/b padding columns and the stripe tail are never read by anything — the run-list-gated zerocheck skips dead blocks, the lincheck kernel is handed `useful_bits` — so they are no longer cleared when the union certifies that (`dead_padding_unread`); z's suffix keeps its memset. (ii) The flush stages eight consecutive groups and writes each chunk-column as one 1 KB burst instead of 128-byte scatters at a 4 MB stride. (iii) The BLAKE3 builder's super-group loop joins the P+E work queue.

Mechanism: (i) is proven rather than argued: a guard test poisons both regions on one of two otherwise identical proves and pins the proofs byte-identical. The 58 GB/s was never the store roofline; it was the roofline for one 128-byte store per page. (ii) keeps 96–384 KB of staging in L2 (768 KB spills and gives the win back).

Result: (i) witgen -10.9 (4/4) and -3.2 ms (8/8) on the micro-bench, -22.4 ms end to end (4/6; more than the two skips account for — 441 MB fewer dirty pages ahead of the commit). (ii) ST 351.6 $\to$ 339.4 (-12), MT -3 to -6. (iii) -2.1 ms MT (6/6). In-prove witgen 64 $\to \approx$35 ms MT.

Two findings frame what is left. The padding memset for z *cannot* be skipped, and the reason is completeness, not soundness: at full utilization the compaction is the identity, so q *is* the padded buffer and the committed polynomial includes the padding while the zerocheck and lincheck claims are computed as if it were zero — forcing the skip makes the honest prover's own proof fail at `PcsOpen(Ligerito)`, the predicted site. (Padding is written entirely prover-side, so it can never affect what a verifier accepts.) And §8.4 was re-examined against the new generator: the part of it that pays — the three streams' zero tails — is exactly what `dead_padding_unread` now skips; the constant regions inside the useful data are worth $\approx$ 1 ms, and a provenance-tagged witness pool was built end to end and found structurally unable to hit (the buffers return through the pool in a different class than they are taken from). What remains is $\approx$ 15 ms of required stores at the burst rate, 18 of BLAKE3 compute, and 13 for the stripe: either the answer or the bus.

9.5 The sparse tail's ladder and the fold-level entry

Baseline: The AG/sparse tail ran one round per pass over the live prefix: a fold-1 entry at $2^{26} \to 2^{25}$ (43 ms MT, 209 ST) then rounds of geometrically shrinking cost. Per pass the lookahead ladder of §4.2 loses on the entry (+16%) and wins on every steady pass (-19%, -21%).

Improvement: (i) The prefix ladder: fold-2 passes with the deferred-challenge state on the compact buffer, "one scaling per row" (pre-scale the four a values by eq so the eight products carry the weight as single unreduced multiplies: 16 $\to$ 12 per position). (ii) The fold-level entry: the skip$\to$multilinear fold forms the round-0/1 lookahead sums itself, NEON-resident off the byte-dots of §9.2, so the tail's first pass is already a fold-2 over the full 48M and there is no entry pass.

Mechanism: Transcript-identical (the round-0 message falls out of the sums). The entry's cost was first priced by *adding* the accumulation to the fold's existing Horner message (+19 ms) and declared dead; in a fused fold the sums *replace* that message, and the fair cost is the difference: +5 ms with the vector-resident formation.

Result: Ladder -4.5 ms MT (2/3); entry -10 ms MT (3/3), -12 ST; tail 62 $\to$ 45–47.

The mis-pricing is worth recording as a rule: when pricing a fusion, *replace, don't add.* A related null closes the subject: the friendly-Horner tail kernel is 2–3$\times$ slower than the ladder on this path and is never dispatched.

9.6 The statistics ladder in the open

Baseline: The open's level-0 basis is a sum of T=2 rank-one eq tensors (the merged transport's seeded point and the one level-0 OOD point), and the six initial rounds bind exactly the six lane-block bits. The prover nonetheless swept the 2^{25} witness for the combine's prime and lookahead (46 ms MT), swept it again for the OOD (20), then folded it through five passes with the virtual basis evaluated per output (63): the same structure paid for three times.

Improvement: Write the index as $u = e \cdot d + h$; each term is $s_t\, \mathrm{blk}_t[e]\, \mathrm{within}_t[h]$. Every level-0 round message is a function of the *block statistics* $G_t[e] = \sum_h f[e,h]\, \mathrm{within}_t[h]$

— 64 values per term — and the six array folds compose into one 64-term fold $f'[h] = \sum_e \mathrm{eq}(r,e) f[e,h]$ straight into $\mathbb{F}_{2^{256}}$. One sweep computes both terms' statistics (two multiplies per element); the OOD evaluation and β come from the 64-entry tables; the switch basis is $\sum_t s_t\, \mathrm{blk}_t[0]\, \mathrm{within}_t[h]$.

Mechanism: Same field elements in the same transcript slots, so proof bytes are identical; an exact oracle test at small geometry. This is the challenge tree's "direct fold", which the port audit had filed as dead by protocol because their *code* was tied to the 100-bit basis open. The algebra is not: it needs a rank-one basis and lane-major block rounds, which the $\mathbb{F}_{2^{256}}$ ladder has. Re-derived, not ported.

Result: Leaf open 298.9 → 204.6 ms; in-prove open −80 to −107, prove −115 to −138 ms (3/3), best prove 760.6 → 630.3. The total moves ≈ 35 ms more than the open bucket because the ladder no longer allocates fold 1's two 256 MB transients and returns the witness right after its one fold: less pool churn.

Three follow-ups each removed a pass the ladder had left. The seeded term's statistics are the merged sumcheck's own folded witness after $\log n - \text{initial\_}k = 19$ of its 25 rounds (the same coordinate convention, dead blocks honest zeros), so the sweep covers the OOD term alone: 10.4 → 6.7 ms (3/3). The code switch's three conversions (split, promote, split the basis: 80 MB of fresh allocations) are gone because the fold writes its output directly in the switch's split form and the basis is materialized already split: switch 7.7 → 0.7 ms (3/3). And the switch's message promotion had been a *serial* map over 2^{20} elements into a fresh allocation beside a parallel neighbour: 9.5 → 3–4 ms, with a lesson attached — the first version also routed the switch's buffers through the scratch pool, which made the switch faster and the *next* fold 3 → 21–62 ms: at a 24-entry cap the most-populated-class eviction policy threw out buffers the fold reuses. Adding pool traffic is not a local change (§9.12).

9.7 The weight build with the column factor baked into fold tables

Baseline: The merged transport's weight $W[d] = \sum_i \varphi_i(\mathrm{eq}_{\mathrm{row},i}[r] \cdot \mathrm{eq}_{\mathrm{col},i}[c])$ cost, per element and per RS claim, one field multiply by the hoisted column factor and then the 16-lookup byte-table map φ_i.

Improvement: The column factor is constant across a column segment and $x \mapsto \varphi_i(\mathrm{eq}_{\mathrm{col}}[c]\cdot x)$ is $\mathbb{F}_2$-linear, so its byte decomposition *is* a fold table: bake $\Psi_c = \varphi_i \circ (\mathrm{eq}_{\mathrm{col}}[c]\cdot)$ per column (92 live × 64 KB per claim, ≈ 1 ms MT, parallel over columns) and pay the sixteen lookups only.

Mechanism: The same field elements, so W and everything downstream are identical (pins pass). The same principle as the virtual basis and the statistics: never materialize an eq tensor, and never multiply by a factor that a table can absorb.

Result: W build MT 32.2 → 25.4 ms (3/3), ST 227 → 167.

9.8 Mixed-field intake for the level OODs and the induced bases

Baseline: The recursive levels' OOD bases ($\mathrm{eq}(z,\cdot)$, 2^{20} at level 1) and the induced query basis (2^{19}, transported across the split as the pairs $(B_j, 0), (0, B_j)$) are base-field valued, and so is the table they meet; the intake promoted each to $\mathbb{F}_{2^{256}}$ (a 32 MB allocation and write), formed the message with $\mathbb{F}_{2^{256}} \times \mathbb{F}_{2^{256}}$ products (three $\mathbb{F}_{2^{128}}$ multiplies each), split the induced basis into a second 64 MB array, and glued with mixed products into both limbs. These buckets scaled only 3.5× ST→MT: page faults on the fresh promotions.

Improvement: A pending-basis form (`Base`, `PresplitBase`): the OOD evaluation and its (u_0, u_2) are three $\mathbb{F}_{2^{128}}$ products per pair in one sweep; the presplit message is $u_0 = \sum f_0 B_j$, $u_2 = (S, S)$ with $S = \sum(f_0{+}f_1)B_j$ (two products per pair, since $u \cdot B = (0, B)$ and $(1{+}u)s = (s, s)$ for base s); the glue adds $B_j\beta$ into the right limb.

Mechanism: Same field elements; no promotion, no split array; the promoted intake deleted.

Result: Level OODs 10.4 → 4.6–5.6 ms MT (35.9 → 11.7 ST), induce 10–11 → 7 MT (36.2 → 23 ST); open $-16/-17$; best prove 601.4 ms.

9.9 The level-0 Merkle was hashing every leaf on the generic path

Baseline: §7.1's eight-message kernel was dispatched only for leaf sizes 64/128/256/512/1024 ("leaf sizes are $16 \ll \log$ batch, so only powers of two arise" — true before the integer-lane commit). The union commits 46 lanes at $m{=}32$: a 736-byte leaf, so every one of the 2^{20} leaves went to the generic single-stream hasher. The commit attribution had called this "at the hash roofline for rate 1/4" on a 2 GB codeword; the level-0 rate is 1/2 and the stored codeword is 736 MiB, so the Merkle was running at 6.3 GB/s — a third of the kernel's rate.

Improvement: The kernel takes the last block's true length (a short final block is zero-padded, as BLAKE3 specifies) and the batcher accepts any single-chunk leaf (1–1024 bytes), tail leaves on the generic path.

Mechanism: Same chunk chaining values, same roots (pins pass; 24 unit tests over the new sizes, 736 among them). The misattribution is the lesson: a "roofline" claim has to be checked against the actual byte count.

Result: Merkle MT 116.7 → 39.8 ms (3/3), 19.8 GB/s on the all-core pool (the challenge tree's rate is 21.6); commit 232.9 → 154.4; prove −67 to −90; ST Merkle 941 → 314, commit 1640 → 1008.

This fix answered a question the numbers had been hiding: with the AG skip so much cheaper than RS round 1, why had commit + zerocheck + lincheck not fallen? Before the fix the sum was 405–415 ms, the same as the pre-merge 401 — AG's saving (RS round 1 165 ms all-in → AG 48) had been cancelled by round 1 no longer having an idle commit window to hide under, and by the Merkle at a third of its rate. After it the sum is ≈ 340.

9.10 Block-first merged sumcheck from column statistics

Baseline: The merged transport (the ring-switching note, *docs/capacity-free-ring-switching.tex*) materialized its weight (25 ms) and ran a 25-round product sumcheck over the 2^{25} domain (46 ms) before the open.

Improvement: Under a full-height column prefix the weight is a sum of rank-one terms (256 per RS claim: a column bit-bank $T[c][b] = \sum_r \mathrm{eq}(z_{\mathrm{row}}, r)\,\mathrm{bit}_b(q[c, r])$ against a column factor), so the block coordinates are bound first from block statistics obtained by a Frobenius transform of the bit banks, and only the last rounds touch the rows.

Mechanism: Every weight term masked to the live prefix (the verifier's $\widehat{W}$ is the zero-tail extension); ρ rotated into coordinate order in prover, verifier and the recursive walker. The C claim's bit bank is one extra fold of the lincheck stripe at the C row point (13.6 ms, outside the open).

Result: Merged sumcheck 46 → 14 ms, weight build gone; open 137 → 109 ms MT, prove ≈ -15 ms net of the C fold (3/3).

The full fusion (the weight as the open's level-0 basis, no merged sumcheck at all) was built to the native verifier and parked: the residual side of the recursive verifier does not twist for free, and its +8.5k rows per chain child buy a one-percent leaf gain. Fusing the C fold into the lincheck's own sweep (a two-point stripe kernel with 32-byte sum-table entries and sixteen accumulators) was built, bit-identical, and measured a wash (-0.3 ms): the stripe kernel is core-throughput-bound at ≈ 1.4 lookups per cycle per core (85.9 ms on one thread, 14.4 on eight, 37 GB/s), not bandwidth-bound, so a second point costs its own load and XOR per stripe byte and shares only the addressing. With the q' fold and the ladder's seed sweep at the multiply rate, stage 1's remaining ≈ 13 ms of transport tax is protocol-shaped.

9.11 Lane-fill tiling and first-prove latency

Baseline: The commit's lane-major fill tiled at 64 positions, and the tile length is the per-lane read burst: 1 KB, with t lanes whose sources sit 8 MiB apart. The prefetcher never establishes a stream across 61 far-separated 1 KB bursts; a pure copy ran at 9 GB/s.

Improvement: A size-aware tile, tile $= (d/(4\,\text{threads}))$ clamped to $[64, 1024]$ positions, keeping at least four tiles per worker.

Mechanism: Longer bursts per lane; the same words to the same addresses.

Result: ST fill 136 $\to$ 94 ms (-31%; 1024 positions in the sweep); MT a wash — at eight workers the pass is bandwidth-saturated at any burst length.

Two latency items, invisible to a warm minimum and important to a one-shot prover: the statement-binding digest (a BLAKE3 hash of every type's sparse matrices, $\approx$ 21M nonzeros) was materialized inside the *first* prove, 1.3 s; it now joins the setup's warm-up (`bind statement` 1297 $\to$ 0.01 ms on the first prove). And the first open in a process is ≈ 100 ms colder than the warm one (279 against 181 at the time): a pool eviction that unmaps a multi-hundred-megabyte buffer, plus cold recursive folds.

9.12 Scratch-pool retention: the count was the constraint

Baseline: The scratch pool retained at most 24 buffers, evicting from the most-populated size class. At steady state the m=32 prove cycles ≈ 25 (a dozen small ladder and query buffers plus the witness $\times 3$, the codeword, the zerocheck's four $2^{25.5}$-word giants and the fold outputs), so the pool overflowed on every prove and evicted the giants during the open's gives; the next zerocheck missed four times and re-mapped ≈ 3 GB of fresh pages — $\approx$ 197k page reclaims per prove.

Improvement: Retention 24 $\to$ 48.

Mechanism: Peak RSS is unchanged (7.2–7.9 GB in every configuration — the buffers exist during every prove anyway; retention only adds idle footprint). A warm-first (LIFO) tie-break was tried in the same experiment and is harmful in both settings; the 2026-07-28 byte-budget experiment recorded in the source had moved nothing because it ranked eviction by bytes at the same *count* cap.

Result: Same binary, eight-prove steady state, four alternating pairs: prove -55 to -96 ms (mean -68, 4/4), zerocheck+lincheck -39, misses 45 $\to$ 12 per nine proves, page reclaims 2.0M$\to$ 1.0M; best prove 444.5, then 438.3 ms on a quiet box.

Two follow-ups closed the subject. There is no other leak: steady-state faults are 660 per prove and RSS is flat over eight proves; the fault storms this desktop shows under load are the memory compressor reclaiming the retired witness's pages (the "witgen cold-page lottery": the same phase reads 3–4 ms with its pages resident and 44–100 when they were compressed, which is why an A/B compares only the phase it changed). And wiring the pool's large buffers with `mlock` measured nothing on a quiet box (4 pairs, mixed signs within ±10 ms on min, median and max) while adding ≈ 63k faults per prove for the wiring walks.

Summary

Measurement protocol. Results before 2026-08-27 include proof-of-work grinding; later entries marked *grind-free* zero the grinding bits on both trees (`FLOCK_NO_GRIND=1`; the library verifiers stay honest, so such proofs fail verification by design) — nonce-search luck otherwise adds variance paired runs cannot cancel. Grind-free figures form their own baseline family. Sub-3-ms phase-local effects are certified by pairing the phase line itself rather than bucket or total times. From Section 9 on, the shape is the m=32 union at production settings, measured same-binary where a knob allows it, otherwise alternating saved binaries; steady-state figures take the minimum over eight proves in one process, and a phase whose pages the memory compressor may have reclaimed (witness generation, on this desktop) is compared only when it is the phase under test.

Rejected alternatives. The kept changes are one side of a 17-experiment record; the failures shaped them. On this core, re-encoding fixed work never paid: pairing multiplies through a memory-interfaced kernel (+10%), a four-register structured load (+12%, microcoded), three-way XOR fusion (+1.8%), an 8× smaller gather table (−3.6% regression), and deferring an already-cheap reduction (0%) all failed, while every win either removed work, added independent work in flight, or removed register-file boundary crossings. In the commit-phase NTT, rewriting the butterfly multiply as a 3-PMULL Karatsuba with a q-resident interface regressed 58%: the generic butterfly already inlines the latency-optimal binius kernel, and six PMULLs with no repack beat three with one, in situ and not only in the microbenchmark. Two further structural transplants measured neutral end-to-end and were reverted despite making the zerocheck *bucket* look better (hoisting the challenge-independent prep under the commit; the lookahead double-round — which first lost 7–15% in the zerocheck tail and was rejected here, until the cross-tree kernel anatomy showed the loss belonged to the scalar accumulator, not the technique: re-kerneled it is now §4.2). The commit-phase campaign added measured nulls of its own: pairing the AB prep with the Merkle stage on the theory that BLAKE3 (integer SIMD) and PMULL use disjoint ports (both issue on the NEON pipes; the Merkle arm went 55 → 150 ms beside the prep); hashing leaves *inside* the NTT's deep tasks (serializes BLAKE3 behind PMULL on the same core — the win required the E-core *pipeline* of §8.2); a 16-stream NEON fused-4 top (2.5× worse: prefetch breakdown at huge strides) and a first fused-3 whose per-lane scatter stores paid read-for-ownership on the fresh destination — radix-8 only won as §8.3's staged-store kernel, a reminder that a null adjudicates one implementation in one architecture, not the idea. A pooled induce densify buffer also measured null (the right-sizing copy-out ate the fault savings at 32 MB scale), and the whole-prove E-core pass kept only one of three sites: hetero tiles on the open's boundary sweep and recursive-commit leaves measured negative — on this host the pattern needs ≳30 ms compute-bound windows to amortize its helper threads.

Rejected alternatives, after the merge. Section 9's twelve kept changes sit beside about twenty measured refusals, and several of them are the more instructive record. *Scheduling nulls*: hoisting round-1 AB prep under the union's commit moved its buckets exactly as designed (round 1 120 → 55) and the commit paid it all back (+52, 0/6) — the NTT is at 50 GB/s with its top tiles already on P+E and the Merkle at the hash roofline on the all-core pool, so the window has no idle capacity to hoist into; the same holds for AG round 1, which is more hoistable in principle. The lane-major streaming commit measured 6/12 at +2 ms, i.e. zero. The E-core queue that pays on round 1 and the witness builder loses +24 ms (3/3) on the skip→mlv fold and +3 on the statistics sweep: E-cores help only flat, compute-bound loops, never a phase at the bus. *Kernel re-encodings*: 16-bit gather tables (4 gathers from 4 MiB instead of 8 from 32 KiB) regressed 21–25%; a Karatsuba accumulate for AG round 1 has no reduction to defer; structured-b for AG has a 2% ceiling; stripe-C for AG derives cleanly onto batch-major addresses and costs more multiplies than it removes transposes; two transposed-NTT layers per pass in the induced-basis builder were worth 0.3 ms (16 MB, SLC-resident); a tiled NEON radix-8 in the recursive commits' NTT gained 0.5 on the top pass and lost 1.0 on the deep one. *Fusions priced honestly*: an alternating skip/double-fold schedule on the merged product sumcheck (two variants, both byte-identical) saved traffic on paper and lost on the wall (W build +5, or ST +26); having witness generation compute round 1's challenge-free half is negative because round 1's reads are not exposed (38.5 of its 46 ms is compute) and the intermediate would cost more to re-read than the 8 ms it saves; interleaving the witness builder's stores with its compute was built, bit-identical, and measured nothing; the two-point stripe fold (§9.10) shares only the addressing. *Structural*: a provenance-tagged witness pool cannot hit; a four-level Fast/Slim ladder saves 12.9 KB of proof natively but the recursive verifier's row envelope cannot hold the 16× residual; the full fusion of the transport into the open is parked for the same verifier. *Memory*: `mlock` on the pool, and a LIFO pool tie-break, both refuted. The one recurring mistake was pricing a fusion by adding its new work to the old (§9.5); the one recurring success was reading the cost accounting before writing a kernel (§9.9, §9.4, §9.12).

Where the pipeline stands. Three reference points, all on this machine, m=32, grind-free (multi-thread / single-thread): the retired direct pipeline at the branch's pre-merge tip, 458.9 ms / 3.28 s with a 437 KB proof; the union pipeline as merged, 853.8 / 5.25; the union pipeline now, 438.0 / 2.91 with a 462 KB proof. Against the direct pipeline the commit is −42% (§9.9, §7.1), zerocheck + lincheck +16% (a faster zerocheck, but the union lincheck and the C-claim bank are new work), and the open 3.6× (105 against 30 ms: two ring-switched claims and the merged transport where the old path had one direct open — protocol, not implementation). The independent challenge tree (the Yukon *flock-challenge* frontier: $\mathbb{F}_{2^{128}}$ statement, SHA-256 Merkle, 19-bit fold grinding, no union, plus a Metal GPU commit and zerocheck fold) proves the same 2^{18} compressions in 266 ms here as the benchmark runs it, 384–407 with its GPU switched off, and 2.86 s single-threaded: the CPU work is equal, its CPU-only edge is parallel efficiency (an E-core helper pool and keep-alive spinners), and its real edge is the GPU.

Section	Kind	Pha
§3.1	representation	rou
§2.4	work removal	ro
§2.3	ILP	ro
§2.1	arithmetic	ro
§2.2	structural	ro
§3.2	representation	rou
§4.1	pass fusion	rounds
§6.1	work removal	open co
§6.2	work removal	open (m=32): −34%
§6.3	pass fusion	open (m=
§4.2	pass fusion	zc tail (m=
§4.3	representation, pass fusion	zc r2..5 (m=32
§4.4	work removal, scheduling	commit join / zc r1 (m=32
§6.4+§6.5	work removal	open (m=
§6.8	pass fusion	open (m=32
§6.6+§6.7	scheduling, pass fusion	open (m=
§8.2	architecture	commit+prep (m=32): −3% wa
§8.3	kernel, scheduling	commit+prep (m=
§1.1	work removal	witne
§5.1	work removal	linchec
§7.1	ILP	BLAKE3 merkle
on the union protocol, m=32 (2^{18} compressions), grind-free		
§9.1	representation	round
§9.2	kernel reuse	AG fold: −7.5%
§9.3	dispatch	pro
§9.4	work removal, store pattern	witness gen: 64 → ≈35 ms 8T;
§9.5	pass fusion	zc tail: −14.5 m
§9.6	work removal (algebra)	open: −80 to −107 ms 8T; prove
§9.7	representation	W build: −7 m
§9.8	representation	level OODs + induce: −9 m
§9.9	dispatch	merkle: 116 → 39 ms 8T
§9.10	protocol transport	open: 137
§9.11	store pattern	lane
§9.12	allocation	prove: −68
round 1 cumulative		
round 2 cumulative (§3.1+§3.2)		
rounds 3+ cumulative		
PCS open cumulative (§6.1+§6.2, m=30 ST / m=32 ST)		
end-to-end, all seven zerocheck changes		
whole campaign on the old protocol, m=32 BLAKE3 (grind-free, certified final table)		8T 723.9 → 454.
union protocol, m=32 BLAKE3 (grind-free), from the merge to now		8T 853.8 → 438.0 ms (1.95×); ST